

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch16_transformers

AYuSh

Contents

Chapter 16: Transformers	3
--------------------------------	---

Chapter 16: Transformers

Chapter 15 ended with attention rescuing a bottlenecked seq2seq model — and a hint that once attention does the remembering, the recurrence is dead weight. The transformer (Vaswani et al., 2017, "Attention Is All You Need") is that hint taken literally: delete the RNN, keep attention, and add back just enough machinery — positional encodings for the order the recurrence used to carry, feedforward blocks for per-position computation, residuals and normalization for depth — to make a trainable architecture. What the trade buys is everything the RNN could not offer: every position attends to every other in one hop (path length 1 for gradients, vs $|i - j|$ hops), and all positions compute in parallel during training (the RNN's sequential bottleneck, gone). What it costs is $O(n^2)$ attention and the return of "where am I?" as an explicit design problem. Interviews live on this chapter: derive scaled dot-product attention, explain multi-head, compare positional encodings, place the layer norms, mask the future, cache the keys, and know what FlashAttention actually does.

Everything is measured: the variance argument behind $\sqrt{d}$ scaling, with softmax's Jacobian dying eight orders of magnitude when the scale is dropped (Listing 1); a multi-head module with full analytic backward, and a task where one head provably cannot do the work of two (Listing 2); positional encodings as executable properties — permutation equivariance to 10^{-15} , the sinusoidal shift-is-a-rotation identity, RoPE's relative-only dot products, ALiBi's per-head distance horizons (Listing 3); a causal-masking morality play in which the unmasked model reaches training loss 0.0003 by cheating and collapses at generation (Listing 4); exact per-layer gradients through 24-block pre-norm and post-norm stacks (Listing 5); a KV cache verified exact to 10^{-15} with measured speedups (Listing 6); FlashAttention's online softmax reproducing full attention to 3×10^{-16} without ever materializing the score matrix (Listing 7); and a complete decoder-only transformer — multi-head attention, pre-norm blocks, learned embeddings, Adam — gradient-checked end to end and solving at 1.000/1.000 the length-13 reversal that Chapter 15's seq2seq failed at 0.522/0.000 (Listing 8).

Self-attention: queries, keys, values

Strip attention to its logic: every position emits a **query** ("what am I looking for?"), a **key** ("what do I contain, as advertised?"), and a **value** ("what do I actually deliver?") — three learned linear projections of the same input, $Q = XW_Q$, $K = XW_K$, $V = XW_V$. Position i scores every position j by dot product $q_i \cdot k_j$, softmaxes the scores into weights, and returns the weighted average of values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This is Chapter 15's Luong attention promoted to *self*-attention — the sequence attending to itself — with two upgrades. First, separating K from V decouples *addressing* from *content*: what makes a position findable need not be what it contributes. Second, the $\sqrt{d_k}$. The interview derivation: if q and k have independent components with zero mean and unit variance, then $q \cdot k = \sum_{i=1}^d q_i k_i$ has variance d — it grows linearly with dimension.

Listing 1 measures exactly that: variance 4, 64.5, 1023.5 at $d = 4, 64, 1024$, and exactly 1.0 after dividing by $\sqrt{d}$. Unscaled at $d = 512$, the softmax saturates: max weight 1.0000, row entropy 0.055 (vs 2.26 scaled), and the softmax Jacobian's mass collapses from 1.78 to 1.3×10^{-8} — eight orders of magnitude of gradient, gone. This is Chapter 12's saturation story in new clothes, and the mirror image of Chapter 15's Listing 7, where scores too *small* left attention uniformly smeared and a sharpening was needed: softmax has a temperature sweet spot, and $\sqrt{d_k}$ is the dimension-independent way to sit in it. The listing closes by gradient-checking the whole op — through the softmax Jacobian $\alpha \odot (d\alpha - \alpha^T d\alpha)$ — to 2.4×10^{-9} .

Multi-head attention

One attention head produces, per position, *one* convex combination of value vectors — one softmax, one "place to look." Many relations need several simultaneous looks: syntax and coreference, the noun and its modifier. **Multi-head attention** runs H attention functions in parallel on learned low-dimensional projections (each head d_{model}/H -dimensional), concatenates the H context vectors, and mixes them with an output projection W_O . The compute is roughly that of one full-width head — the point is not capacity but *multiplicity of addressing*.

Listing 2 makes "one head cannot look two places" a theorem-by-experiment. The task: from a sequence of feature-value triples, report the value at the position of maximum feature 1 *and* the value at the position of maximum feature 2 — two unrelated retrievals. Same total attention width (8 dims), same training budget: one 8-dimensional head lands at test MSE 1.074 — indistinguishable from the predict-zero baseline of 1.0, i.e. it learned *nothing* — while two 4-dimensional heads reach 0.045. The failure is structural, not statistical: a single softmax can average the two target positions but cannot deliver two *different* selections through one convex combination, and averaging destroys both answers. The full analytic backward (through both heads' softmaxes into W_K, W_V , the query vectors, and W_O) gradient-checks at 3×10^{-10} . Worth volunteering in interviews: trained heads specialize into interpretable roles (positional heads, syntactic heads, and the *induction heads* that Chapter 20's in-context learning story leans on) — and head-pruning studies show many heads are redundant after training, so multiplicity matters most during learning.

Positional encodings

Permute self-attention's input and the output permutes identically — Listing 3 verifies $\text{attn}(PX) = P\text{attn}(X)$ to 1.8×10^{-15} . Attention is a *set* operation; word order is invisible. Every transformer therefore injects position, and the four standard answers are an interview staple.

Sinusoidal (the original): $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$,

$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$ — each dimension pair a sinusoid at a geometrically spaced frequency, added to the embedding. The property the authors bet on: for any offset k , $PE(pos + k)$ is a *fixed linear function* (a block rotation) of $PE(pos)$, so relative offsets are linearly readable. Listing 3 builds the rotation matrix from k alone and maps $PE(p) \rightarrow PE(p + k)$ for all 200 positions with max error 2.2×10^{-14} ; dot products $PE(p) \cdot PE(p + 5)$ come out 23.504 at $p = 0, 50, 150$ — a function of offset, not absolute position. No parameters, defined for any length (though extrapolation quality is another matter).

Learned absolute: a trainable vector per position (BERT, GPT-2). Simple, and lets the model shape its own geometry — but positions beyond the training length simply don't exist, a hard extrapolation ceiling.

RoPE (rotary): instead of *adding* position to the residual stream, *rotate* q and k — each 2-dim slice by angle $pos \cdot \theta_i$. Rotating q_m by $m\theta$ and k_n by $n\theta$ makes their dot product depend on $n - m$ only: relative position enters exactly where positions interact (the score), absolute position appears nowhere. Listing 3 verifies it: q at positions 3/53/503 dotted with k seven ahead gives -12.257587 all three times. RoPE is the modern default (Llama-family, most open models), and stretching its angles — position interpolation, NTK/YaRN scaling — is how pretrained models get their context windows extended.

ALiBi: no embedding at all — add a *linear penalty* $-m_h \cdot |i - j|$ to attention scores, slope m_h geometric across heads. Each head becomes a different recency horizon: Listing 3's four heads show mean attended distance 5.9, 12.6, 16.1, 17.1 of a possible 31 — a panel of short- and long-sighted heads. Because the bias formula works at any distance, ALiBi's claim to fame is length extrapolation beyond the training window.

Encoder, decoder, encoder-decoder — and the causal mask

Three ways to assemble the blocks, one bit of information flow deciding everything. An **encoder** (BERT) lets every position attend everywhere — bidirectional, Chapter 15's BiRNN idea done with attention — ideal for understanding tasks (classification, NER, retrieval embeddings) and incapable of generation, because it was never trained to predict without seeing the answer's neighborhood. A **decoder** (GPT) applies a **causal mask**: position t attends only to positions $\leq t$, implemented by adding $-\infty$ above the diagonal of the score matrix before the softmax (zeros out future weights exactly). That

single triangular matrix is what makes teacher forcing legitimate at scale: all n next-token predictions train *in parallel* in one pass, each one honestly blind to its own answer. An **encoder-decoder** (T5, the original machine-translation transformer) keeps both: bidirectional encoder over the source, causal decoder over the target, joined by **cross-attention** — decoder queries against encoder keys/values, Chapter 15's Bahdanau mechanism in its final form.

Listing 4 stages what happens if the mask is forgotten. A next-token model trained *unmasked* on period-2 sequences reaches training loss 0.0003 and teacher-forced accuracy 1.000 — it simply attends to position $t+1$ and copies the answer that is sitting in its input. Generate free-running, where the future genuinely doesn't exist yet, and it collapses to 0.590. The causal model posts an honest 0.65 training loss (the first token after the prefix really is unpredictable), and free-runs at 1.000. The unmasked model's perfect training loss is the tell — an interviewer's favorite debugging scenario, and the transformer's version of Chapter 4's leakage: the label was in the features.

Inside the block: FFN, residuals, and layer-norm placement

A transformer block is attention plus a position-wise **feedforward network** — two linear layers with a nonlinearity, hidden width typically $4d$ — each sublayer wrapped in a residual connection and a LayerNorm (Chapter 13's per-position normalizer; BatchNorm's batch coupling is wrong for variable-length sequences). The division of labor: attention *moves* information between positions; the FFN, applied identically and independently at every position, *transforms* it — two-thirds of the parameters live there, and the mechanistic-interpretability literature reads FFN layers as key-value memories storing facts and features.

Where the LayerNorm sits is a real decision with a famous failure mode. **Post-norm** (the 2017 original): $x \leftarrow \text{LN}(x + \text{sublayer}(x))$ — normalize *after* the residual add, so the norm sits *inside* the gradient's path and every backward pass through the block gets rescaled. **Pre-norm** (GPT-2 onward): $x \leftarrow x + \text{sublayer}(\text{LN}(x))$ — the residual stream runs clean from loss to embeddings, an uninterrupted identity highway (Chapter 14's residual insight, kept pristine), with sublayers as normalized side-branches. Listing 5 builds both at depth 24 with full analytic backward (gradient-checked to 5×10^{-11}) and measures $\|\nabla W_Q\|$ per layer at initialization: the post-norm stack's gradients span nearly *seven orders of magnitude* from layer 1 to layer 24, while pre-norm spans barely one. No single learning rate serves layers seven decades apart — that is why the original transformer needed its careful warmup schedule, and why pre-norm (which trains without warmup) became the default. The honest trade-off to volunteer: post-norm, once successfully trained, often edges out pre-norm in final quality (pre-norm's late layers can go underused as the residual stream's norm grows), which is why warmup-plus-post-norm and hybrids never fully died.

The KV cache

Autoregressive generation recomputes nothing about the past if you store it. At step t the only *new* row of attention is the new token's query against all previous keys; those keys and values were computed at their own steps and never change (the causal mask guarantees the past is unaffected by the future). The **KV cache** stores every layer's K and V as they are produced; each generation step then computes one query, appends one key and value, and attends once — no mask even needed, since the cache *is* exactly the visible past. Listing 6 verifies exactness (cached step vs full recompute: 1.8×10^{-15} over 48 tokens) and measures the cost: generating T tokens by full recompute is $O(T^3)$ total in attention (T passes of $O(T^2)$), with cache $O(T^2)$ — speedups $2.5\times$, $4.1\times$, $9.1\times$ at $T = 64, 128, 256$ and growing.

The price is memory, and at scale it *is* the serving bottleneck: cache size $= 2 \times \text{layers} \times n \times d_{kv} \times \text{bytes}$, per sequence in the batch. A 7B-class model at long context carries gigabytes of cache per stream. The remedies are current interview material: **multi-query attention** (all heads share one K/V head), **grouped-query attention** (GQA — a few K/V groups; Llama-2-70B onward), cache quantization (8- or 4-bit), and **PagedAttention** (vLLM) allocating cache in virtual-memory-style pages to kill fragmentation. Prompt processing remains parallel (one full pass fills the cache); generation is sequential — the RNN's curse returning through the back door, per token rather than per layer.

Attention complexity and efficient attention

Self-attention costs $O(n^2d)$ time and — naively — $O(n^2)$ memory for the score matrix; the FFN costs $O(nd^2)$. At short lengths the FFN dominates; past $n \approx d$ the quadratic takes over, and long context becomes an engineering discipline of its own.

FlashAttention (Dao et al., 2022) is the answer interviewers most want dissected, and its key claim is routinely misstated: it is *exact* attention, not an approximation — it reduces *memory traffic*, not FLOPs. The trick is the **online softmax**: process K, V in tiles, maintaining per query a running maximum m , normalizer ℓ , and weighted accumulator; when a new tile raises the max, rescale what has been accumulated by $e^{m_{old} - m_{new}}$ and continue. The $n \times n$ matrix is never formed — only score tiles that live in fast on-chip SRAM instead of being written to and re-read from HBM (attention is memory-bandwidth-bound, so avoided traffic is the whole win). Listing 7 implements the streaming recurrence and matches full attention to 3.2×10^{-16} — bit-level exact — with peak score storage $n \times \text{tile}$ instead of n^2 ($8\times$ less at $n = 512$, and the ratio is n/tile : it grows with sequence length).

Approximations trade exactness for asymptotics. **Sparse attention** (Longformer, BigBird): fixed patterns — local windows, strided, a few global tokens — giving $O(nw)$. **Sliding-window attention** (Mistral): each layer attends only w back, but reach

compounds across layers exactly like CNN receptive fields — Listing 7's boolean information-flow product shows position 0's influence growing w per layer, 4, 8, 12, ..., 24 over six layers of window 4. **Linear attention** (Performer, linear transformers) replaces softmax with kernel feature maps for $O(n)$, historically at a quality cost. And below the algorithmic layer sit MQA/GQA (shrink the KV, Listing 6's discussion) and, outside attention entirely, state-space models (Mamba) — recurrence's modern comeback, $O(n)$ inference with parallel training, the pendulum Chapter 15 flagged.

Capstone: a decoder-only transformer, end to end

Listing 8 assembles the chapter: token and learned positional embeddings, two pre-norm blocks of 4-head causal self-attention and $4 \times$ FFN, a final LayerNorm and unembedding, cross-entropy on next-token prediction, the complete analytic backward through every piece — checked at once through attention, FFN, embedding, and positional parameters to 5×10^{-9} — and Adam. The task is a deliberate callback: reverse a length-13 sequence, formatted as [source, SEP, target] with loss on the target positions. Chapter 15's fixed-vector seq2seq managed token accuracy 0.522 and whole-sequence accuracy 0.000 on exactly this length; the attention-augmented version needed a hand-tuned score sharpening to succeed. The transformer trains in 1,500 Adam steps and free-runs at **1.000 token, 1.000 whole-sequence** — no bottleneck to collapse (every generated position attends directly to its source token, path length 1), no temperature surgery (the $\sqrt{d}$ scaling was the fix, built in), and training parallel across all 13 predictions per sequence. The three RNN diseases of Chapter 15 — vanishing credit over distance, the fixed-vector bottleneck, sequential training — are not mitigated here; they are absent by construction. That is why this architecture ate the field, and why Parts IV's chapters (pretraining, alignment, inference, RAG, agents) all assume it.

Code implementations

Listing 1 — Scaled dot-product attention -- and why the sqrt(d) matters

```

"""Listing 1: scaled dot-product attention from scratch -- and why the sqrt(d)
matters."""
import numpy as np
rng = np.random.default_rng(0)

def softmax(z, axis=-1):
    z = z - z.max(axis=axis, keepdims=True)
    e = np.exp(z); return e / e.sum(axis=axis, keepdims=True)

def attention(Q, K, V, scale=True):
    d = Q.shape[-1]
    S = Q @ K.T / (np.sqrt(d) if scale else 1.0)    # scores (n_q, n_k)
    A = softmax(S, axis=-1)                          # attention weights
    return A @ V, A                                  # context, weights

# 1) variance of raw dot products grows linearly with d

```



```

for d in [4, 64, 1024]:
    q = rng.normal(size=(20000, d)); k = rng.normal(size=(20000, d))
    dots = (q * k).sum(1)
    print(f"d={d:5d}: var(q.k) = {dots.var():8.1f}  var(q.k/sqrt(d)) = {(dots/
np.sqrt(d)).var():.3f}")

# 2) consequence: softmax saturation and gradient death without scaling
n, d = 16, 512
Q = rng.normal(size=(n, d)); K = rng.normal(size=(n, d)); V = rng.normal(size=(n, d))
for scale in [True, False]:
    _, A = attention(Q, K, V, scale)
    ent = -(A * np.log(A + 1e-30)).sum(1).mean() # entropy of attn rows (max
ln16=2.77)
    # gradient of softmax wrt scores: J = diag(a) - a a^T; its norm ~ how trainable
    row = A[0]; J = np.diag(row) - np.outer(row, row)
    print(f"scale={str(scale):5s}: max weight {A.max():.4f}  row entropy {ent:.3f}  ||
softmax Jacobian|| {np.abs(J).sum():.2e}")

# 3) gradient check the whole attention op (loss = sum of context)
def loss(Qf):
    C, _ = attention(Qf.reshape(n, d), K, V); return C.sum()
def grads():
    C, A = attention(Q, K, V)
    dC = np.ones_like(C)
    dA = dC @ V.T
    dS = A * (dA - (A * dA).sum(-1, keepdims=True)) # softmax Jacobian, rowwise
    return dS @ K / np.sqrt(d)
g = grads(); eps = 1e-6; worst = 0
for idx in [(0,0), (5,100), (15,511)]:
    keep = Q[idx]; Q[idx] = keep+eps; Lp = loss(Q.ravel()); Q[idx] = keep-eps; Lm =
loss(Q.ravel()); Q[idx] = keep
    num = (Lp-Lm)/(2*eps)
    worst = max(worst, abs(num-g[idx])/max(1e-12, abs(num)+abs(g[idx])))
print(f"attention gradient check on Q: worst relative error {worst:.2e}")

```

Output:

```

d=   4: var(q.k) =      4.0  var(q.k/sqrt(d)) = 1.001
d=  64: var(q.k) =     64.5  var(q.k/sqrt(d)) = 1.008
d=1024: var(q.k) =    1023.5  var(q.k/sqrt(d)) = 0.999
scale=True : max weight 0.6388  row entropy 2.259  ||softmax Jacobian|| 1.78e+00
scale=False: max weight 1.0000  row entropy 0.055  ||softmax Jacobian|| 1.34e-08
attention gradient check on Q: worst relative error 2.37e-09

```

Listing 2 — Multi-head attention with full backward -- one head cannot look two places

```

"""Listing 2: multi-head attention with full backward -- why one head cannot look two
places."""
import numpy as np
rng = np.random.default_rng(1)

def softmax(z):
    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e / e.sum(-1,
keepdims=True)

# task: x_i = [f1_i, f2_i, v_i]; target = [v at argmax f1, v at argmax f2]
# model: H learned query vectors; head h: scores_i = q_h . (X Wk_h), ctx_h = sum a_i (X
Wv_h); out = Wo [ctx_1..ctx_H]
def make(n=8):
    f = rng.normal(size=(n, 2)); v = rng.normal(size=(n, 1))
    X = np.concatenate([f, v], 1)

```

```

y = np.array([v[f[:,0].argmax(),0], v[f[:,1].argmax(),0]])
return X, y

def run(H, steps=20000, lr=0.03, dh=4):
    r = np.random.default_rng(42)
    D = 3
    Wk = r.normal(0, .5, (H,D,dh)); Wv = r.normal(0, .5, (H,D,dh)); q = r.normal(0, .5,
(H,dh))
    Wo = r.normal(0, .5, (H*dh,2))
    def fwd(X):
        Ks = X @ Wk; Vs = X @ Wv # (H,n,dh)
        S = np.einsum('hnd,hnd->hn', Ks, q) / np.sqrt(dh)
        A = softmax(S) # (H,n)
        C = np.einsum('hn,hnd->hd', A, Vs) # (H,dh)
        out = C.reshape(-1) @ Wo
        return out, (X, Ks, Vs, A, C)
    def step(X, y):
        out, (X, Ks, Vs, A, C) = fwd(X)
        err = 2*(out - y); L = ((out-y)**2).sum()
        dC = (Wo @ err).reshape(H, dh)
        dWo = np.outer(C.reshape(-1), err)
        dA = np.einsum('hd,hnd->hn', dC, Vs)
        dVs = np.einsum('hn,hnd->hnd', A, dC)
        dS = A * (dA - (A*dA).sum(-1, keepdims=True))
        dKs = np.einsum('hn,hnd->hnd', dS, q) / np.sqrt(dh)
        dq = np.einsum('hn,hnd->hd', dS, Ks) / np.sqrt(dh)
        dWk = np.einsum('ni,hnd->hid', X, dKs); dWv = np.einsum('ni,hnd->hid', X, dVs)
        return L, (dWk, dWv, dq, dWo)
    # gradient check once
    X, y = make(); _, g = step(X, y); eps = 1e-6; worst = 0
    for arr, garr, idx in [(Wk, g[0], (0,1,2)), (q, g[2], (0,3)), (Wo, g[3], (2,1))]:
        keep = arr[idx]; arr[idx] = keep+eps; Lp, _ = step(X,y); arr[idx] = keep-eps;
    Lm, _ = step(X,y); arr[idx] = keep
        num = (Lp-Lm)/(2*eps); worst = max(worst, abs(num-garr[idx])/max(1e-12,
abs(num)+abs(garr[idx])))
    for s in range(steps):
        X, y = make(); L, g = step(X, y)
        for P, G in zip((Wk, Wv, q, Wo), g): P -= lr*np.clip(G, -1, 1)
    test = [step(*make())[0] for _ in range(500)]
    return worst, float(np.mean(test))

for H, dh in [(1, 8), (2, 4)]: # SAME total head dim = 8
    worst, mse = run(H, dh=dh)
    print(f"H={H} (dh={dh}, total 8): grad check {worst:.2e} test MSE {mse:.3f}")
print(f"variance of target (predict-zero baseline): {np.var([make()[1] for _ in
range(2000)]):.3f}")

```

Output:

```

H=1 (dh=8, total 8): grad check 2.48e-09 test MSE 1.074
H=2 (dh=4, total 8): grad check 3.18e-10 test MSE 0.045
variance of target (predict-zero baseline): 0.996

```

Listing 3 — Positional encodings: permutation equivariance, sinusoidal shifts, RoPE, ALiBi

```

"""Listing 3: positional encodings -- permutation equivariance, sinusoidal shifts,
RoPE, ALiBi."""
import numpy as np
rng = np.random.default_rng(2)

def softmax(z):

```

```

    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e / e.sum(-1,
keepdims=True)
def attn(X, Wq, Wk, Wv, bias=0.0):
    Q, K, V = X@Wq, X@Wk, X@Wv
    A = softmax(Q @ K.T / np.sqrt(Q.shape[-1]) + bias)
    return A @ V

# 1) self-attention is permutation-EQUIVARIANT: attn(PX) == P attn(X)
n, d = 10, 16
X = rng.normal(size=(n, d)); Wq, Wk, Wv = (rng.normal(0, .5, (d,d)) for _ in range(3))
P = np.eye(n)[rng.permutation(n)]
print(f"max |attn(PX) - P attn(X)| = {np.abs(attn(P@X,Wq,Wk,Wv) -
P@attn(X,Wq,Wk,Wv)).max():.2e}")
    f" -> order is invisible without positional information")

# 2) sinusoidal PE: PE(pos+k) is a LINEAR function of PE(pos) (a fixed rotation)
def sinpe(pos, d=64):
    i = np.arange(d//2); ang = pos / 10000**(2*i/d)
    out = np.zeros(d); out[0::2] = np.sin(ang); out[1::2] = np.cos(ang); return out
D = 64; pes = np.stack([sinpe(p, D) for p in range(200)])
k = 7
R = np.zeros((D, D))
rotation, built from k alone
for i in range(D//2):
    th = k / 10000**(2*i/D); c, s = np.cos(th), np.sin(th)
    R[2*i:2*i+2, 2*i:2*i+2] = [[c, -s], [s, c]]
err = np.abs(pes[:193] @ R - pes[k:200]).max()
print(f"sinusoidal: fixed rotation matrix R_k (k={k}) maps PE(p)->PE(p+k) for ALL p:
max err {err:.2e}")
d05 = [pes[p] @ pes[p+5] for p in (0, 50, 150)]
print(f"PE(p).PE(p+5) at p=0/50/150: {d05[0]:.3f} / {d05[1]:.3f} / {d05[2]:.3f}
(depends ~only on offset)")

# 3) RoPE: rotate q and k by position-dependent angles; the DOT depends only on
relative offset
def rope(x, pos, base=10000):
    d = len(x); i = np.arange(d//2); th = pos / base**(2*i/d)
    c, s = np.cos(th), np.sin(th)
    y = x.copy(); y[0::2] = x[0::2]*c - x[1::2]*s; y[1::2] = x[0::2]*s + x[1::2]*c
    return y
q, kv = rng.normal(size=64), rng.normal(size=64)
for m, nn in [(3, 10), (53, 60), (503, 510)]:
    absolute positions
    print(f"RoPE q_at_{m:3d} . k_at_{nn:3d} (offset 7) = {rope(q,m) @ rope(kv,nn):.
6f}")

# 4) ALiBi: add -slope*distance to causal scores; heads get geometric slopes
slopes = [2*(-8*(h+1)/4) for h in range(4)]
S = rng.normal(size=(32, 32))
row 31
dist = np.arange(32)[::-1].astype(float)
for h, m in enumerate(slopes):
    A = softmax(S[31] - m*dist)
    eff = (A * dist).sum()
distance
print(f"ALiBi head {h}: slope {m:.4f} mean attended distance {eff:5.1f} / 31")

```

Output:

```

max |attn(PX) - P attn(X)| = 1.78e-15 -> order is invisible without positional
information
sinusoidal: fixed rotation matrix R_k (k=7) maps PE(p)->PE(p+k) for ALL p: max err
2.23e-14
PE(p).PE(p+5) at p=0/50/150: 23.504 / 23.504 / 23.504 (depends ~only on offset)
RoPE q_at_ 3 . k_at_ 10 (offset 7) = -12.257587
RoPE q_at_ 53 . k_at_ 60 (offset 7) = -12.257587

```

```

RoPE q_at 503 . k_at 510 (offset 7) = -12.257587
ALiBi head 0: slope 0.2500 mean attended distance 5.9 / 31
ALiBi head 1: slope 0.0625 mean attended distance 12.6 / 31
ALiBi head 2: slope 0.0156 mean attended distance 16.1 / 31
ALiBi head 3: slope 0.0039 mean attended distance 17.1 / 31

```

Listing 4 — Causal masking -- the unmasked next-token model learns by cheating

```

"""Listing 4: causal masking -- the unmasked next-token model 'learns' by cheating."""
import numpy as np
rng = np.random.default_rng(3)

V, n, d = 8, 12, 24
def softmax(z):
    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e / e.sum(-1,
keepdims=True)
def make():
    x = np.empty(n, int); x[:2] = rng.integers(0, V, 2)
    for t in range(2, n): x[t] = x[t-2] # rule: token repeats with
period 2
    return x
E = rng.normal(0, 1, (V, d)) # frozen random embeddings
pe = rng.normal(0, 1, (n, d)) # frozen random positional
vectors

def run(causal, steps=4000, lr=0.05):
    r = np.random.default_rng(7)
    Wq, Wk, Wv = (r.normal(0, .4, (d,d)) for _ in range(3)); Wu = r.normal(0, .4, (d,V))
    mask = np.where(np.tril(np.ones((n,n)))==1, 0.0, -1e9) if causal else 0.0
    def fwd(x):
        X = E[x] + pe[:len(x)]
        Q, K, Vv = X @ Wq, X @ Wk, X @ Wv
        A = softmax(Q @ K.T / np.sqrt(d) + (mask if causal else 0.0))
        H = A @ Vv
        return X, A, H, softmax(H @ Wu)
    def step(x):
        X, A, H, P = fwd(x)
        tgt = x[1:] # predict x[t+1] from position t
        L = -np.log(P[np.arange(n-1), tgt] + 1e-12).mean()
        dH = np.zeros((n, V)); dH[np.arange(n-1), tgt] = -1
        dlog = (P + dH) # softmax-CE grad, rows 0..n-2
        dlog[-1] = 0; dlog /= (n-1)
        dWu = H.T @ dlog; dHh = dlog @ Wu.T
        dA = dHh @ (X@Wv).T; dV = A.T @ dHh
        dS = A * (dA - (A*dA).sum(-1, keepdims=True))
        dQ = dS @ (X@Wk) / np.sqrt(d); dK = dS.T @ (X@Wq) / np.sqrt(d)
        return L, (X.T@dQ, X.T@dK, X.T@dV, dWu)
    for s in range(steps):
        L, g = step(make())
        for Pm, G in zip((Wq, Wk, Wv, Wu), g): Pm -= lr*np.clip(G, -1, 1)
    # teacher-forced accuracy (full sequence given) vs free-running generation from 2-
token prefix
    tf, fr = [], []
    for _ in range(300):
        x = make()
        _,_,_,P = fwd(x); tf.append((P[:n-1].argmax(1) == x[1:]).mean())
        gen = list(x[:2])
        for t in range(2, n):
            _,_,_,P = fwd(np.array(gen + [0]*(n-len(gen)), int))
    # pad; causal ignores pad, unmasked SEES it
    gen.append(P[len(gen)-1].argmax())
    fr.append((np.array(gen[2:]) == x[2:]).mean())
    return L, np.mean(tf), np.mean(fr)
for causal in [False, True]:

```

```
L, tf, fr = run(causal)
print(f"causal={str(causal):5s}: train loss {L:.4f}  teacher-forced acc {tf:.3f}
free-running acc {fr:.3f}")
```

Output:

```
causal=False: train loss 0.0003  teacher-forced acc 1.000  free-running acc 0.590
causal=True : train loss 0.6474  teacher-forced acc 0.925  free-running acc 1.000
```

Listing 5 — Pre-norm vs post-norm: exact per-layer gradients at depth 24

```
"""Listing 5: pre-norm vs post-norm -- exact gradient per layer at init, depth 24, full
analytic backward."""
import numpy as np
rng = np.random.default_rng(4)
n, d, depth = 8, 32, 24
WR = np.random.default_rng(99).normal(0, d**-.5, d)

def softmax(z):
    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e / e.sum(-1,
keepdims=True)
def ln_f(x):
    mu = x.mean(-1, keepdims=True); xc = x - mu
    var = (xc**2).mean(-1, keepdims=True); inv = 1/np.sqrt(var + 1e-6)
    return xc*inv, (xc, inv)
def ln_b(dy, cache):
    xc, inv = cache; D = xc.shape[-1]
    dx = inv*(dy - dy.mean(-1, keepdims=True) - xc*inv**2*(dy*xc).mean(-1,
keepdims=True))
    return dx

def init(seed):
    r = np.random.default_rng(seed)
    return [dict(Wq=r.normal(0,d**-.5,(d,d)), Wk=r.normal(0,d**-.5,(d,d)),
Wv=r.normal(0,d**-.5,(d,d)), Wl=r.normal(0,d**-.5,(d,4*d)),
W2=r.normal(0,(4*d)**-.5,(4*d,d))) for _ in range(depth)]

def attn_f(h, L):
    Q, K, V = h@L['Wq'], h@L['Wk'], h@L['Wv']
    A = softmax(Q @ K.T / np.sqrt(d)); return A@V, (h, Q, K, V, A)
def attn_b(dout, cache, L, g):
    h, Q, K, V, A = cache
    dA = dout @ V.T; dV = A.T @ dout
    dS = A*(dA - (A*dA).sum(-1, keepdims=True))
    dQ = dS @ K/np.sqrt(d); dK = dS.T @ Q/np.sqrt(d)
    g['Wq'] += h.T@dQ; g['Wk'] += h.T@dK; g['Wv'] += h.T@dV
    return dQ@L['Wq'].T + dK@L['Wk'].T + dV@L['Wv'].T

def fwd_bwd(X, layers, pre):
    caches = []
    for L in layers:
        c = {}
        if pre:
            h, c['ln1'] = ln_f(X); a, c['at'] = attn_f(h, L); X1 = X + a
            h2, c['ln2'] = ln_f(X1); z = h2@L['W1']; r = np.maximum(0, z)
            c['ffn'] = (h2, z, r); X = X1 + r@L['W2']; c['X1'] = X1
        else:
            a, c['at'] = attn_f(X, L); X1, c['ln1'] = ln_f(X + a)
            z = X1@L['W1']; r = np.maximum(0, z); c['ffn'] = (X1, z, r)
            X, c['ln2'] = ln_f(X1 + r@L['W2'])
        caches.append(c)
```

```

y = X @ WR # fixed linear readout
loss = ((y - 1)**2).mean(); dX = np.outer(2*(y - 1)/len(y), WR)
grads = [dict((k, np.zeros_like(v)) for k, v in L.items()) for L in layers]
for L, c, g in zip(layers[::-1], caches[::-1], grads[::-1]):
    h2_or_X1, z, r = c['ffn']
    if pre:
        dr = dX @ L['W2'].T; g['W2'] += r.T@dX
        dz = dr*(z > 0); g['W1'] += h2_or_X1.T@dz
        dX1 = dX + ln_b(dz@L['W1'].T, c['ln2'])
        dh = attn_b(dX1, c['at'], L, g)
        dX = dX1 + ln_b(dh, c['ln1'])
    else:
        dS2 = ln_b(dX, c['ln2']) # through the post-LN
        dr = dS2 @ L['W2'].T; g['W2'] += r.T@dS2
        dz = dr*(z > 0); g['W1'] += h2_or_X1.T@dz
        dX1 = dS2 + dz@L['W1'].T
        dS1 = ln_b(dX1, c['ln1'])
        dX = dS1 + attn_b(dS1, c['at'], L, g)
return loss, grads

# gradient check (depth-3 stack, both variants)
Xs = rng.normal(size=(n, d))
for pre in [False, True]:
    ls = init(1)[:3]
    _, gr = fwd_bwd(Xs, ls, pre); eps, worst = 1e-4, 0
    for li, key, idx in [(0, 'Wq', (3,5)), (1, 'W2', (10,7)), (2, 'Wv', (0,0))]:
        W = ls[li][key]; keep = W[idx]
        W[idx] = keep+eps; Lp, _ = fwd_bwd(Xs, ls, pre); W[idx] = keep-eps; Lm, _ =
fwd_bwd(Xs, ls, pre); W[idx] = keep
        num = (Lp-Lm)/(2*eps)
        worst = max(worst, abs(num-gr[li][key][idx])/max(1e-12, abs(num)+abs(gr[li][key][idx])))
    print(f"{'pre' if pre else 'post'}-norm backward gradient check: worst rel err {worst:.2e}")

X0 = rng.normal(size=(n, d))
for pre in [False, True]:
    layers = init(0)
    _, gr = fwd_bwd(X0, layers, pre)
    norms = [np.linalg.norm(g['Wq']) for g in gr]
    print(f"{'pre' if pre else 'post'}-norm ||dWq||: layer1 {norms[0]:.2e} layer8 {norms[7]:.2e} "
          f"layer16 {norms[15]:.2e} layer24 {norms[-1]:.2e} (last/first = {norms[-1]/norms[0]:.1e})")

```

Output:

```

post-norm backward gradient check: worst rel err 4.98e-11
pre-norm backward gradient check: worst rel err 6.33e-11
post-norm ||dWq||: layer1 1.10e+00 layer8 4.15e-02 layer16 1.52e-04 layer24
5.38e-07 (last/first = 4.9e-07)
pre-norm ||dWq||: layer1 2.33e+01 layer8 6.60e+00 layer16 2.50e+00 layer24 2.08e+00
(last/first = 8.9e-02)

```

Listing 6 — The KV cache: exactness and the cost of generation

```

"""Listing 6: the KV cache -- exactness and the quadratic-vs-cubic generation cost."""
import numpy as np, time
rng = np.random.default_rng(5)

d, H = 64, 4; dh = d // H
Wq, Wk, Wv, Wo = (rng.normal(0, d**-.5, (d, d)) for _ in range(4))
def softmax(z):

```

```

    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e / e.sum(-1,
    keepdims=True)

def full_layer(X):
    """recompute attention for ALL positions (causal)"""
    n = len(X)
    Q = (X@Wq).reshape(n, H, dh); K = (X@Wk).reshape(n, H, dh); V = (X@Wv).reshape(n,
    H, dh)
    mask = np.where(np.tril(np.ones((n, n))) == 1, 0.0, -1e9)
    out = np.empty((n, H, dh))
    for h in range(H):
        A = softmax(Q[:, h] @ K[:, h].T / np.sqrt(dh) + mask)
        out[:, h] = A @ V[:, h]
    return out.reshape(n, d) @ Wo

def cached_step(x_t, cache):
    """one new token: compute its q, APPEND its k,v, attend over the cache only"""
    q = (x_t@Wq).reshape(H, dh)
    cache['K'].append((x_t@Wk).reshape(H, dh)); cache['V'].append((x_t@Wv).reshape(H,
    dh))
    K = np.stack(cache['K']); V = np.stack(cache['V']) # (t, H, dh)
    out = np.empty((H, dh))
    for h in range(H):
        A = softmax(q[h] @ K[:, h].T / np.sqrt(dh)) # no mask needed:
    cache IS the past
    out[h] = A @ V[:, h]
    return out.reshape(d) @ Wo

# 1) exactness: last row of the full computation == the cached step, token by token
X = rng.normal(size=(48, d)); cache = {'K': [], 'V': []}; worst = 0
for t in range(48):
    worst = max(worst, np.abs(full_layer(X[:t+1])[-1] - cached_step(X[t],
    cache)).max())
print(f"cached vs full recompute over 48 generated tokens: max abs diff {worst:.2e}")

# 2) cost of generating T tokens: no cache = sum of full passes (O(T^3) total); cache =
O(T^2)
for T in [64, 128, 256]:
    X = rng.normal(size=(T, d))
    t0 = time.perf_counter()
    for t in range(1, T+1): full_layer(X[:t]) # regenerate
    everything each step
    t_no = time.perf_counter() - t0
    cache = {'K': [], 'V': []}; t0 = time.perf_counter()
    for t in range(T): cached_step(X[t], cache)
    t_yes = time.perf_counter() - t0
    kv_mb = 2 * T * d * 8 / 1e6
    print(f"T={T:3d}: no-cache {t_no:6.3f}s  cache {t_yes:6.3f}s  speedup {t_no/t_yes:
    5.1f}x  KV memory {kv_mb:.2f} MB/layer")

```

Output:

```

cached vs full recompute over 48 generated tokens: max abs diff 1.78e-15
T= 64: no-cache 0.009s  cache 0.003s  speedup 2.5x  KV memory 0.07 MB/layer
T=128: no-cache 0.042s  cache 0.011s  speedup 3.8x  KV memory 0.13 MB/layer
T=256: no-cache 0.274s  cache 0.031s  speedup 9.0x  KV memory 0.26 MB/layer

```

Listing 7 — Efficient attention: FlashAttention's online softmax and sliding-window reach

```

"""Listing 7: efficient attention -- FlashAttention's online softmax, tile by tile, and
sliding-window reach."""

```



```

import numpy as np
rng = np.random.default_rng(6)

def softmax(z):
    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e / e.sum(-1,
keepdims=True)

# 1) FlashAttention's core: process K/V in TILES, maintaining running (max, normalizer,
weighted sum).
# The n x n attention matrix is never materialized -- yet the result is EXACT.
n, d, tile = 512, 64, 64
Q = rng.normal(size=(n, d)); K = rng.normal(size=(n, d)); V = rng.normal(size=(n, d))
ref = softmax(Q @ K.T / np.sqrt(d)) @ V # the O(n^2)-memory
reference

m = np.full(n, -np.inf); l = np.zeros(n); acc = np.zeros((n, d))
for j0 in range(0, n, tile): # stream over key/value
    tiles
    S = Q @ K[j0:j0+tile].T / np.sqrt(d) # (n, tile) -- only a tile
of scores
    m_new = np.maximum(m, S.max(1)) # running row max
    corr = np.exp(m - m_new) # rescale factor for what
we accumulated
    p = np.exp(S - m_new[:, None])
    l = l*corr + p.sum(1)
    acc = acc*corr[:, None] + p @ V[j0:j0+tile]
    m = m_new
flash = acc / l[:, None]
print(f"online-softmax (tile {tile}) vs full attention: max abs diff {np.abs(flash -
ref).max():.2e}")
print(f"peak score storage: full {n*n} floats vs tiled {n*tile} floats ({n//tile}x
less)")

# 2) sliding-window attention: window w per layer, but reach GROWS by w per layer (like
CNN receptive fields)
w, L, N = 4, 6, 64
adj = np.abs(np.arange(N)[:, None] - np.arange(N)[None, :]) <= w # who can attend
whom in ONE layer
reach = np.eye(N, dtype=bool)
for layer in range(1, L+1):
    reach = reach @ adj
# boolean matrix product = information flow
print(f"after layer {layer}: position 0 reaches positions up to {np.where(reach[0])
[0].max()}")

```

Output:

```

online-softmax (tile 64) vs full attention: max abs diff 3.19e-16
peak score storage: full 262144 floats vs tiled 32768 floats (8x less)
after layer 1: position 0 reaches positions up to 4
after layer 2: position 0 reaches positions up to 8
after layer 3: position 0 reaches positions up to 12
after layer 4: position 0 reaches positions up to 16
after layer 5: position 0 reaches positions up to 20
after layer 6: position 0 reaches positions up to 24

```

Listing 8 — A decoder-only transformer from scratch, solving the reversal that broke seq2seq

```

"""Listing 8: a decoder-only transformer from scratch -- solving the reversal that
broke seq2seq."""
import numpy as np

```



```

rng = np.random.default_rng(8)

Vc, T = 8, 13                                # vocab 0..7, source length 13 (ch15's seq2seq
failure point)
SEP = Vc; VOC = Vc + 1                       # one separator token
n = 2*T + 1                                   # [src ... SEP tgt ...]
d, H, depth = 48, 4, 2; dh = d//H

def softmax(z):
    z = z - z.max(-1, keepdims=True); e = np.exp(z); return e / e.sum(-1,
keepdims=True)
def ln_f(x):
    mu = x.mean(-1, keepdims=True); xc = x - mu
    inv = 1/np.sqrt((xc**2).mean(-1, keepdims=True) + 1e-6); return xc*inv, (xc, inv)
def ln_b(dy, c):
    xc, inv = c
    return inv*(dy - dy.mean(-1, keepdims=True) - xc*inv**2*(dy*xc).mean(-1,
keepdims=True))

r = np.random.default_rng(0)
params = dict(E=r.normal(0, .5, (VOC,d)), P=r.normal(0, .5, (n,d)), Wu=r.normal(0,d**-.5,
(d,VOC)))
for i in range(depth):
    for w, shp, sc in (('Wq',(d,d),d**-.5),('Wk',(d,d),d**-.5),('Wv',(d,d),d**-.5),
('Wo',(d,d),d**-.5),('W1',(d,4*d),d**-.5),('W2',(4*d,d),
(4*d)**-.5)):
        params[f'{w}{i}'] = r.normal(0, sc, shp)
MASK = np.where(np.tril(np.ones((n,n)))==1, 0.0, -1e9)

def fwd(x, want_cache=False):
    X = params['E'][x] + params['P'][:len(x)]
    caches = []
    for i in range(depth):
        c = {}
        h, c['ln1'] = ln_f(X)
        Q = (h@params[f'Wq{i}']).reshape(-1,H,dh); K =
(h@params[f'Wk{i}']).reshape(-1,H,dh)
        Vv = (h@params[f'Wv{i}']).reshape(-1,H,dh)
        A = np.stack([softmax(Q[:,hh]@K[:,hh].T/np.sqrt(dh) + MASK[:len(x),:len(x)])
for hh in range(H)])
        ao = np.stack([A[hh]@Vv[:,hh] for hh in range(H)], 1).reshape(-1,d)
        c['at'] = (h, Q, K, Vv, A, ao); X1 = X + ao@params[f'Wo{i}']
        h2, c['ln2'] = ln_f(X1); z = h2@params[f'W1{i}']; rr = np.maximum(0,z)
        c['ffn'] = (h2, z, rr); c['X1'] = X1
        X = X1 + rr@params[f'W2{i}']
        caches.append(c)
    hf, lnf = ln_f(X)
    logits = hf@params['Wu']
    return (logits, caches, lnf, hf, x) if want_cache else logits

def step(x, tgt_mask):
    logits, caches, lnf, hf, x = fwd(x, True)
    Pr = softmax(logits)
    tgt = np.roll(x, -1)
    L = -np.log(Pr[tgt_mask, tgt[tgt_mask]] + 1e-12).mean()
    g = {k: np.zeros_like(v) for k, v in params.items()}
    dlog = Pr.copy(); dlog[np.arange(n), tgt] -= 1
    dlog[~tgt_mask] = 0; dlog /= tgt_mask.sum()
    g['Wu'] += hf.T@dlog; dX = ln_b(dlog@params['Wu'].T, lnf)
    for i in range(depth-1, -1, -1):
        c = caches[i]; h2, z, rr = c['ffn']
        drr = dX@params[f'W2{i}'].T; g[f'W2{i}'] += rr.T@dX
        dz = drr*(z>0); g[f'W1{i}'] += h2.T@dz
        dX1 = dX + ln_b(dz@params[f'W1{i}'].T, c['ln2'])
        h, Q, K, Vv, A, ao = c['at']
        dao = (dX1@params[f'Wo{i}'].T).reshape(-1,H,dh); g[f'Wo{i}'] += ao.T@dX1
        dhid = np.zeros_like(h); dQ = np.zeros_like(Q); dK = np.zeros_like(K); dV =
np.zeros_like(Vv)

```

```

    for hh in range(H):
        dA = dao[:,hh]@Vv[:,hh].T; dV[:,hh] = A[hh].T@dao[:,hh]
        dS = A[hh]*(dA - (A[hh]*dA).sum(-1,keepdims=True))
        dQ[:,hh] = dS@K[:,hh]/np.sqrt(dh); dK[:,hh] = dS.T@Q[:,hh]/np.sqrt(dh)
    for nm, dM in [ ('Wq',dQ), ('Wk',dK), ('Wv',dV) ]:
        g[f'{nm}{i}'] += h.T@dM.reshape(-1,d); dhid +=
dM.reshape(-1,d)@params[f'{nm}{i}'].T
    dX = dX1 + ln_b(dhid, c['ln1'])
    np.add.at(g['E'], x, dX); g['P'][:len(x)] += dX
    return L, g

def make():
    s = rng.integers(0, Vc, T)
    return np.concatenate([s, [SEP], s[::-1]]), s

tgt_mask = np.zeros(n, bool); tgt_mask[T:n-1] = True
# predict positions T+1..2T (the reversed tokens)

# gradient check
x, _ = make(); _, g0 = step(x, tgt_mask); eps, worst = 1e-5, 0
for key, idx in [ ('Wq0',(3,5)), ('W21',(10,7)), ('E',(2,9)), ('P',(5,3)) ]:
    W = params[key]; keep = W[idx]
    W[idx]=keep+eps; Lp,_ = step(x,tgt_mask); W[idx]=keep-eps; Lm,_ =
step(x,tgt_mask); W[idx]=keep
    num=(Lp-Lm)/(2*eps); worst=max(worst, abs(num-g0[key][idx])/max(1e-12,abs(num)
+abs(g0[key][idx])))
print(f"full transformer gradient check (attn/FFN/embed/pos): worst rel err {worst:.
2e}")

# train: Adam, minibatch of 8 sequences
mom = {k: np.zeros_like(v) for k, v in params.items()}; vel = {k: np.zeros_like(v) for
k, v in params.items()}
b1, b2, lr = 0.9, 0.999, 2e-3
for it in range(1, 1501):
    acc = {k: np.zeros_like(v) for k, v in params.items()}; Ls = 0
    for _ in range(8):
        x, _ = make(); L, g = step(x, tgt_mask); Ls += L/8
        for k in acc: acc[k] += g[k]/8
    for k in params:
        mom[k] = b1*mom[k] + (1-b1)*acc[k]; vel[k] = b2*vel[k] + (1-b2)*acc[k]**2
        params[k] -= lr*(mom[k]/(1-b1**it))/(np.sqrt(vel[k]/(1-b2**it))+1e-8)
    if it % 500 == 0: print(f"iter {it}: loss {Ls:.4f}")

# free-running generation: feed source + SEP, generate 13 tokens autoregressively
tok_ok, seq_ok = [], []
for _ in range(200):
    x, s = make(); gen = list(x[:T+1])
    for t in range(T):
        logits = fwd(np.array(gen + [0]*(n-len(gen))), int)
        gen.append(int(logits[len(gen)-1].argmax()))
    out = np.array(gen[T+1:]); tok_ok.append((out == s[::-1]).mean());
seq_ok.append((out == s[::-1]).all())
print(f"T=13 reversal, free-running: token acc {np.mean(tok_ok):.3f} whole-sequence
acc {np.mean(seq_ok):.3f}")
print(f"(chapter 15's bottlenecked seq2seq at T=13: token 0.522, whole-sequence
0.000)")

```

Output:

```

full transformer gradient check (attn/FFN/embed/pos): worst rel err 4.97e-09
iter 500: loss 0.0012
iter 1000: loss 0.0005
iter 1500: loss 0.0003
T=13 reversal, free-running: token acc 1.000 whole-sequence acc 1.000
(chapter 15's bottlenecked seq2seq at T=13: token 0.522, whole-sequence 0.000)

```

Interview questions and answers

Q1. Derive scaled dot-product attention. Why exactly $\sqrt{d_k}$?

Project X into $Q = X \cdot W_Q$, $K = X \cdot W_K$, $V = X \cdot W_V$; scores $S = Q \cdot K^T / \sqrt{d_k}$; weights $A = \text{softmax}(S)$ rowwise; output $A \cdot V$. The scale: if q, k components are independent, zero-mean, unit-variance, then $q \cdot k = \text{sum of } d \text{ products}$ has variance d (Listing 1: measured 4/64.5/1023.5 at $d = 4/64/1024$). Dividing by $\sqrt{d}$ restores variance 1 regardless of dimension. Without it, scores at $d = 512$ spread over tens of units, softmax saturates to one-hot (entropy 0.055), and the softmax Jacobian $\text{diag}(a) - a \cdot a^T$ collapses to $\sim 1e-8$ — no gradient, no learning.

Q2. What do Q , K , and V each represent, and why separate K from V ?

Query: what position i is looking for. Key: how position j advertises itself for matching. Value: what j actually contributes once selected. Separating K from V decouples addressing from content — a position can be findable by one criterion (e.g. syntactic role) while delivering a different payload (its semantic vector). With $K = V$ you force the retrieval geometry and the content geometry to coincide, which wastes capacity; the three learned projections let attention implement arbitrary soft key-value lookup.

Q3. Why multiple heads? Give evidence that one head is structurally insufficient for some tasks.

One head emits ONE softmax per position — one convex combination of values, one place to look. Multi-head runs H attentions on d/H -dimensional projections and concatenates, so a position can retrieve several unrelated things simultaneously at the same total cost. Listing 2: retrieve the value at argmax of feature 1 AND at argmax of feature 2 — one 8-dim head lands at MSE 1.07 (== predict-zero baseline, learned nothing) while two 4-dim heads reach 0.045. Averaging two required selections through one softmax destroys both. Post-training, heads specialize (positional, syntactic, induction heads), though pruning studies show many become redundant.

Q4. Why do transformers need positional encodings at all?

Self-attention is permutation-equivariant: $\text{attn}(P \cdot X) = P \cdot \text{attn}(X)$ for any permutation P (Listing 3 verifies to $1.8e-15$). Scores depend only on content dot products, so "the dog bit the man" and "the man bit the dog" produce identical attention structure up to reordering — order is invisible. Position must be injected explicitly: added to embeddings (sinusoidal, learned), rotated into q/k (RoPE), or biased into scores (ALiBi). RNNs never had this problem — order was implicit in the recurrence — which is exactly what got deleted.

Q5. State the sinusoidal encoding and the property it was designed for.

$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{(2i/d)})$, $\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{(2i/d)})$ — dimension pairs are sinusoids at geometrically spaced frequencies, from wavelength $2 \cdot \pi$ to $\sim 10000 \cdot 2 \cdot \pi$. Design property: $\text{PE}(\text{pos}+k)$ is a fixed LINEAR function of $\text{PE}(\text{pos})$ — a block-diagonal rotation depending only on k — so a linear layer can read relative offsets. Listing 3 constructs that rotation from k alone and maps every position with error $2.2e-14$; $\text{PE}(p) \cdot \text{PE}(p+5)$ is 23.504 at $p = 0, 50$, and 150 — offset-dependent, position-independent. Parameter-free and defined for any length, though quality beyond training length still degrades.

Q6. Explain RoPE. Why is rotating better than adding?

RoPE rotates each 2-dim slice of q and k by angle $\text{pos} \cdot \text{theta}_i$ (geometric thetas, as in sinusoidal). Because rotations compose, $(R_m \cdot q) \cdot (R_n \cdot k) = q^T \cdot R_{(n-m)} \cdot k$ — the score depends on the relative offset $n-m$ only; absolute position cancels exactly (Listing 3: q at positions 3/53/503 against k seven ahead gives -12.257587 all three times). Advantages over additive: relative position enters precisely where positions interact (the score), it does not occupy residual-stream capacity, and its angular structure supports context extension by interpolating angles (position interpolation, NTK/YaRN scaling). It is the default in modern open models.

Q7. How does ALiBi work and what is its selling point?

No positional embedding at all: add $-m_h \cdot |i-j|$ to attention scores, with slope m_h geometric across heads. Each head gets a different recency horizon — Listing 3's four heads attend mean distances 5.9/12.6/16.1/17.1 of 31 — a built-in panel from myopic to far-sighted. Selling point: length extrapolation. The bias formula is defined at every distance, so a model trained at length 1024 degrades gracefully at 2048+, where learned-absolute embeddings simply do not exist and sinusoidal ones go out of distribution. Cost: a hard-coded recency prior — retrieval-heavy tasks that need exact far lookback can suffer.

Q8. Compare learned absolute vs sinusoidal vs RoPE vs ALiBi in one pass.

Learned absolute (BERT, GPT-2): maximum flexibility, zero extrapolation — positions past training length are undefined parameters. Sinusoidal: parameter-free, linear-readable offsets, any length representable but extrapolation mediocre in practice. RoPE: relative-only interaction, clean score-level injection, extendable via angle scaling — the modern default. ALiBi: simplest, best zero-shot extrapolation, but imposes recency decay. Interview framing: the trend moved from "where is this token absolutely" toward "how far apart are these two tokens," because attention only ever consumes pairwise structure.

Q9. Encoder vs decoder vs encoder-decoder: architecture, training, use cases.

Encoder (BERT): full bidirectional attention, trained with masked-token objectives; best for understanding — classification, NER, retrieval embeddings; cannot generate coherently. Decoder (GPT): causal mask, next-token training; generation, in-context learning, and — at scale — most things. Encoder-decoder (T5, original transformer): bidirectional encoder over source, causal decoder over target, joined by cross-attention (decoder queries, encoder keys/values); natural for input-output tasks like translation and summarization where the source deserves unrestricted bidirectional reading. Decoder-only won the scale race largely on simplicity: one stack, one objective, every token supervised.

Q10. How is the causal mask implemented, and why -infinity rather than zeroing weights?

Add a matrix with 0 on and below the diagonal and $-1e9$ (effectively $-\infty$) above, to the scores BEFORE softmax: $\exp(-\infty) = 0$, so future positions get exactly zero weight and the remaining weights still sum to 1. Zeroing attention weights AFTER softmax instead would break normalization (rows no longer sum to 1) and still leak future content through the denominator, since $\exp(s_{\text{future}})$ inflates the normalizer of every past-position weight. Masking scores, not weights, is the correct and numerically standard construction.

Q11. A colleague's next-token model shows near-zero training loss but generates garbage. Diagnose.

Classic causal-mask bug — the model attends to position $t+1$ and copies its own target, which is present in the input during teacher-forced training. Listing 4 stages it: unmasked model reaches train loss 0.0003 and teacher-forced accuracy 1.000, then collapses to 0.59 free-running; the masked model posts an honest 0.65 loss and free-runs at 1.000. The tell is training loss BELOW the entropy floor of genuinely unpredictable tokens. Same family: leaky data collation (targets not shifted), bidirectional encoder fine-tuned for generation, or evaluation that feeds gold prefixes and calls it generation quality. It is Chapter 4's leakage, transformer edition.

Q12. Why can transformers train all positions in parallel when RNNs cannot?

The RNN's h_t is a function of $h_{(t-1)}$ — a sequential dependency chain; step t cannot start before $t-1$ finishes, so training walltime scales with sequence length regardless of hardware. Causal attention removes the chain: position t 's output depends on positions $1..t$ only through the mask, and ALL of it is matrix algebra over the full sequence at once — one pass computes n next-token predictions, each honestly blind to its future. Teacher forcing (Chapter 15) makes the n losses independent given the data. Generation remains sequential for both — the transformer pays the sequential price per generated token, which is what the KV cache amortizes.

Q13. Pre-norm vs post-norm: equations, and what actually goes wrong with post-norm.

Post-norm: $x = \text{LN}(x + \text{sublayer}(x))$ — the 2017 original; LN sits INSIDE the gradient path, rescaling every backward pass. Pre-norm: $x = x + \text{sublayer}(\text{LN}(x))$ — the residual stream runs uninterrupted from loss to embeddings; sublayers hang off it as normalized branches. Listing 5, depth 24 at init, exact analytic gradients: post-norm $\|dW_Q\|$ spans $\sim$ seven orders of magnitude across layers (1.1 to $5e-7$) vs about one for pre-norm. No single learning rate serves layers seven decades apart — hence the original's mandatory warmup and the field's migration to pre-norm (GPT-2 onward), which trains without warmup. Caveat worth volunteering: successfully trained post-norm often wins slightly on final quality, so warmup+post-norm and hybrid schemes persist.

Q14. What does the FFN sublayer do that attention doesn't? Where do the parameters live?

Attention MOVES information between positions (its only nonlinearity is the softmax over positions); the FFN — two linear maps with a nonlinearity, hidden width $\sim 4d$, applied identically and independently at each position — TRANSFORMS the gathered information. Roughly two-thirds of a transformer's parameters are FFN. Mechanistic-interpretability work reads FFN layers as key-value memories: first layer's rows are pattern detectors, second layer writes associated content into the residual stream — where much factual knowledge appears to be stored (relevant to editing and distillation questions). No FFN = a stack of weighted averaging that composes into not much: attention alone is low-rank mixing.

Q15. Why LayerNorm rather than BatchNorm in transformers?

BatchNorm normalizes each feature across the BATCH: statistics depend on batch composition, break at batch size 1, need running averages at inference, and are ill-defined across variable-length padded sequences — token position 7 of one sentence has no business sharing statistics with position 7 of another. LayerNorm normalizes across FEATURES per token: batch-independent, length-independent, identical at train and inference, and it plays the optimization role Chapter 13 established (smoother loss surface, larger usable learning rates). RMSNorm — LayerNorm minus the mean-centering — is the cheaper modern variant (Llama-family) with essentially equal quality.

Q16. Give the time and memory complexity of a transformer layer, and identify when each term dominates.

Attention: $O(n^2 \cdot d)$ time (scores $n^2 \cdot d$, weighted sum $n^2 \cdot d$), $O(n^2)$ score memory if materialized. FFN: $O(n \cdot d^2)$ time. Ratio n/d decides: short sequences with wide models are FFN-bound; past $n \sim d$ the quadratic dominates — at $n = 32k$, $d = 4k$ attention is $\sim 8x$ the FFN cost. Also on the exam: KV cache memory $2 \cdot \text{layers} \cdot n \cdot d_{kv}$ per sequence at inference, and FlashAttention removing the $O(n^2)$ MEMORY (not FLOPs) by tiling. RNN comparison: $O(n \cdot d^2)$ sequential vs the transformer's parallel $O(n^2 \cdot d)$ — the trade of compute for parallelism and path length 1.

Q17. Explain the KV cache. Why is it exact, and what does it cost?

During generation, keys and values of past tokens never change (the causal mask means the future cannot affect them), so recomputing them every step is pure waste. Cache every layer's K, V; per new token compute one query, append one k, v, attend once — Listing 6 verifies equality with full recompute to $1.8e-15$ and measures speedup growing with length ($2.5x/4.1x/9.1x$ at $T = 64/128/256$; asymptotically $O(T^3)$ total becomes $O(T^2)$). Cost: memory, $2 \cdot \text{layers} \cdot n \cdot d_{kv}$ bytes per sequence — gigabytes at long context, and THE serving bottleneck: batch size is limited by cache, not weights. Remedies: MQA/GQA, cache quantization, PagedAttention.

Q18. What are multi-query and grouped-query attention, and why do they exist?

They shrink the KV cache. MQA: all H query heads share ONE k/v head — cache drops by $H \cdot x$ (e.g. $32x$) at some quality cost. GQA: middle ground, H query heads share G kv groups (e.g. 32 queries, 8 groups — $4x$ smaller cache), recovering nearly full quality; standard since Llama-2-70B. Why it matters: inference throughput is bound by cache memory and the bandwidth to read it every step, not by FLOPs; halving the cache roughly doubles the servable batch. Note the asymmetry: queries are per-step transient and never cached, so only k/v heads need shrinking.

Q19. Is FlashAttention an approximation? Explain what it actually optimizes and how.

Not an approximation — bit-for-bit exact attention (Listing 7: $3.2e-16$ vs reference). It optimizes MEMORY TRAFFIC, not FLOPs: standard attention writes the n^2 score matrix to HBM and reads it back through softmax and the value product; attention is bandwidth-bound, so that round trip is the cost. FlashAttention tiles K/V, keeps score tiles in on-chip SRAM, and never materializes n^2 — via the online softmax: maintain running max m , normalizer l , and accumulator per query; when a tile raises the max, rescale accumulated quantities by $\exp(m_{\text{old}} - m_{\text{new}})$ and continue. Peak score storage n -tile instead of n^2 . The backward recomputes tiles instead of storing them — trading FLOPs for bandwidth, a net win.

Q20. Write the online-softmax update rule and explain why the rescaling is needed.

Per query, state (m, l, acc) . For each key tile: $m_{\text{new}} = \max(m, \text{max of new scores})$; $\text{corr} = \exp(m - m_{\text{new}})$; $p = \exp(S_{\text{tile}} - m_{\text{new}})$; $l = l \cdot \text{corr} + \text{sum}(p)$; $\text{acc} = \text{acc} \cdot \text{corr} + p \cdot V_{\text{tile}}$; $m = m_{\text{new}}$. Final output acc/l . The rescaling: numerically safe softmax subtracts the row max before exponentiating, but a streaming pass discovers the max incrementally — when a later tile raises it, everything already accumulated was exponentiated against a stale max and is too large by $\exp(m_{\text{old}} - m_{\text{new}})^{-1}$... precisely the corr factor fixes it. Same-max case: $\text{corr} = 1$, plain accumulation. This is Listing 7's loop verbatim, and a popular whiteboard-coding ask.

Q21. Sliding-window attention limits each layer to w tokens back. Is long-range information lost?

Not necessarily — reach compounds across layers exactly like CNN receptive fields (Chapter 14): layer 1 sees w back, layer 2 sees information that already traveled w , so $2w$, and L layers reach $L \cdot w$ (Listing 7's boolean information-flow product: 4, 8, ..., 24 over six layers of window 4). Mistral used $w = 4096$ over 32 layers — 131k theoretical reach. Caveats: the path is multi-hop, so distant credit assignment is diluted (it must survive L intermediate mixings), and exact long-distance retrieval suffers versus full attention — why hybrids keep a few global tokens (Longformer/BigBird) or interleave full-attention layers.

Q22. Where does cross-attention appear, and how does it differ from self-attention?

In encoder-decoder models (and multimodal stacks: text queries attending image encoder outputs; Chapter 24's fusion variants). Queries come from the decoder's stream; keys and values from the ENCODER's output — $Q = Y_{\text{dec}} \cdot W_Q$, $K = X_{\text{enc}} \cdot W_K$, $V = X_{\text{enc}} \cdot W_V$. No causal mask needed (the source is fully known), and the encoder's k/v are computed once per source then reused across all decoding steps — a static KV cache. It is Chapter 15's Bahdanau attention in final form: each target step consults every source position directly; the alignment matrices that fell out there are cross-attention weights here.

Q23. Why did transformers displace RNNs? Give the three-part argument with numbers from these two chapters.

(1) Path length: RNN gradient between positions i, j crosses $|i-j|$ Jacobians — signal decayed to $1.2e-5$ by $T = 100$ (ch15 L2); attention connects any pair in one hop. (2) No bottleneck: seq2seq compressed the source into one vector — token accuracy 0.522, sequence accuracy 0.000 at $T = 13$; the transformer's every position attends the full source — 1.000/1.000 on the same task (Listing 8). (3) Parallelism: RNN training is sequential in T ; causal attention trains all positions at once, so hardware scales with data. The costs accepted: $O(n^2)$, explicit positional machinery, and sequential generation (mitigated by the KV cache).

Q24. What breaks when you gradient-check a full transformer, compared to a single attention layer?

The usual failure sites: (1) the softmax Jacobian $dS = A \cdot (dA - \text{sum}(A \cdot dA))$ — omitting the second term is the classic bug; (2) LayerNorm's backward (the mean and variance paths); (3) residual accumulation — dX must SUM the straight-through and branch gradients at every block; (4) embedding gradients need scatter-add (`np.add.at`), since repeated tokens accumulate; (5) masked positions must contribute exactly zero loss gradient. Practical protocol from Listing 8: check one parameter from each distinct path (attention weight, FFN weight, embedding, positional) in one pass — $5e-9$ worst error — in float64 with central differences, before any training run.

Q25. Estimate the KV cache for a 32-layer model, d_model 4096, GQA with 8 kv heads of dim 128, at 32k context, fp16, batch 16.

Per token per layer: $2 \text{ (K and V)} \cdot 8 \cdot 128 = 2048$ values; fp16 = 4096 bytes = 4 KB. Times 32 layers: 128 KB per token. Times 32k tokens: 4 GB per sequence. Times batch 16: 64 GB — more than the ~14 GB the 7B-class weights themselves take. This arithmetic is why GQA exists (full MHA with 32 heads would be 4x this: 256 GB), why cache quantization to int8 halves it again, and why serving systems (vLLM's PagedAttention) treat cache as the resource to schedule. Being able to produce this number quickly is a strong systems-interview signal.

Q26. You must serve 128k-token documents. Walk through your architecture/serving options.

Model side: RoPE with context extension (position interpolation / NTK / YaRN) if starting from a shorter-context checkpoint; sliding-window or hybrid sparse attention to cut $O(n^2)$ (reach = $L \cdot w$, Listing 7); ALiBi if training from scratch with extrapolation in mind. Serving side: FlashAttention (exact, removes n^2 memory), GQA + cache quantization (Q25's arithmetic at 128k is brutal: ~16 GB/sequence unmitigated), PagedAttention for fragmentation, prefix/prompt caching for shared document contexts, and chunked prefill to bound latency. Honest alternative to volunteer: retrieval (Chapter 24) — if the task needs 2k relevant tokens from the 128k, RAG beats brute-force context both in cost and often in accuracy.

Q27. Why does attention have a "temperature sweet spot"? Connect the $\sqrt{d}$ scaling to ch15's sharpening trick.

Softmax gradient flow dies at both extremes. Scores too LARGE (unscaled $d = 512$, Listing 1): near-one-hot weights, Jacobian $\text{diag}(a) - a \cdot a^T \sim 0$ because a is a vertex of the simplex — gradients $1e-8$, nothing trains. Scores too SMALL (ch15's attention, low-dim scores): near-uniform weights, no differentiation signal, attention never learns where to look — fixed by multiplying scores by 4. Same knob, opposite corrections: transformers DIVIDE by $\sqrt{d}$ because high-dim dot products start too hot; the ch15 model MULTIPLIED because its scores started too cold. General principle: keep score variance ~ 1 at init so the softmax sits where its Jacobian has mass.

Q28. Trace this chapter as a set of measured claims an interviewer could probe.

Dot-product variance grows as d ; $\sqrt{d}$ fixes it ($4/64/1024 \rightarrow 1.0$) and unscaled softmax kills gradients 8 orders (Listing 1). One softmax = one look; two retrievals need two heads (MSE 1.07 baseline vs 0.045, Listing 2). Attention is a set operation (equivariance $1.8e-15$); sinusoidal shift is a rotation ($2.2e-14$), RoPE is relative-only (-12.257587 thrice), ALiBi is per-head horizons (Listing 3). Unmasked training-loss perfection is leakage ($0.0003 \rightarrow 0.59$ free-run vs causal 1.000, Listing 4). Post-norm spans 7 gradient decades at depth 24, pre-norm ~ 1 — warmup vs not (Listing 5). KV cache is exact ($1.8e-15$) and turns $O(T^3)$ generation into $O(T^2)$ (Listing 6). FlashAttention is exact tiling ($3.2e-16$), not approximation; window reach compounds $L \cdot w$ (Listing 7). And the full scratch decoder solves ch15's 0.522/0.000 failure at 1.000/1.000 (Listing 8) — the RNN's three diseases absent by construction.